

LeetCode 646 - Maximum Length of Pair Chain

Authored by: Arpit Raj, LNMIIT Jaipur | Date: October 1, 2026

THEORY & LOGIC

Basically koi bhi overlapping interval hata dena he. The goal is to find the maximum number of intervals we can select without any overlap.

- We store the intervals in a structure (pos, type, idx) where start -> type 0 and end -> type 1.
- Then we sort by position. If position is the same, sort by type. (**Classical line sweep**).
- We keep track of the currently selected interval (using curr).
- If we witness an overlapping interval, **we consider the one which finishes first** cause it will leave more room for other intervals. (If pairs[idx][1] < pairs[curr][1], update curr = idx).
- If it is of end type and the index of both current and end are the same, it means one interval is **sorted** (successfully added to the chain). We update our answer and reset curr = -1 for upcoming intervals.

C++ Source Code

```
class Solution {
public:
    int findLongestChain(vector<vector<int>>& pairs) {
        //basically koi bhi overlapping interval hata dena he

        int n=pairs.size();
        int sorted=0;

        vector<pair<pair<int,int>,int>> ev;
        for(int i=0;i<n;i++){
            ev.push_back({{pairs[i][0],0},i});
            ev.push_back({{pairs[i][1],1},i});
        }
    }
};
```

```

        sort(ev.begin(),ev.end(),[](auto &a,auto &b){
            if(a.first.first==b.first.first){
                return a.first.second<b.first.second;
            }

            return a.first.first<b.first.first;
        });

        int curr=-1;
        for(auto e:ev){
            int pos=e.first.first;
            int type=e.first.second;
            int idx=e.second;

            if(type==0){
                if(curr==-1){
                    curr=idx;
                }
                else{
                    if(pairs[idx][1]<pairs[curr][1]){
                        curr=idx;
                    }
                }
            }

            else{
                if(idx==curr){
                    sorted++;
                    curr=-1;
                }
            }
        }
        return sorted;
    }
};

```